

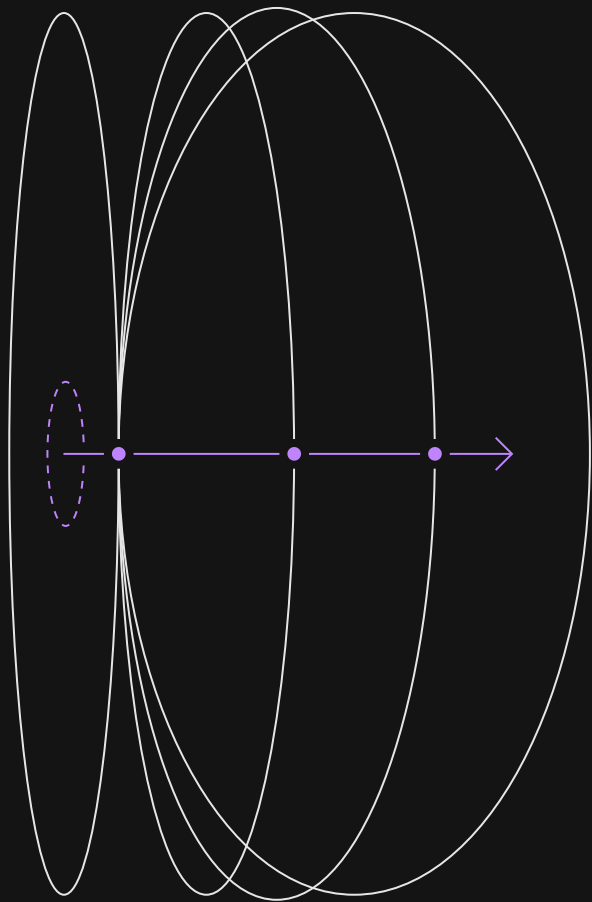


ЦЕНТРАЛЬНЫЙ
УНИВЕРСИТЕТ

Git и GitLab

РАЗРАБОТКА НА PYTHON

НЕДЕЛЯ 1



План лекции



Узнаем, что такое система контроля версий и зачем она нужна



Рассмотрим, как работать с Git — инициализация репозитория, коммиты, ветки, слияние, rebase;



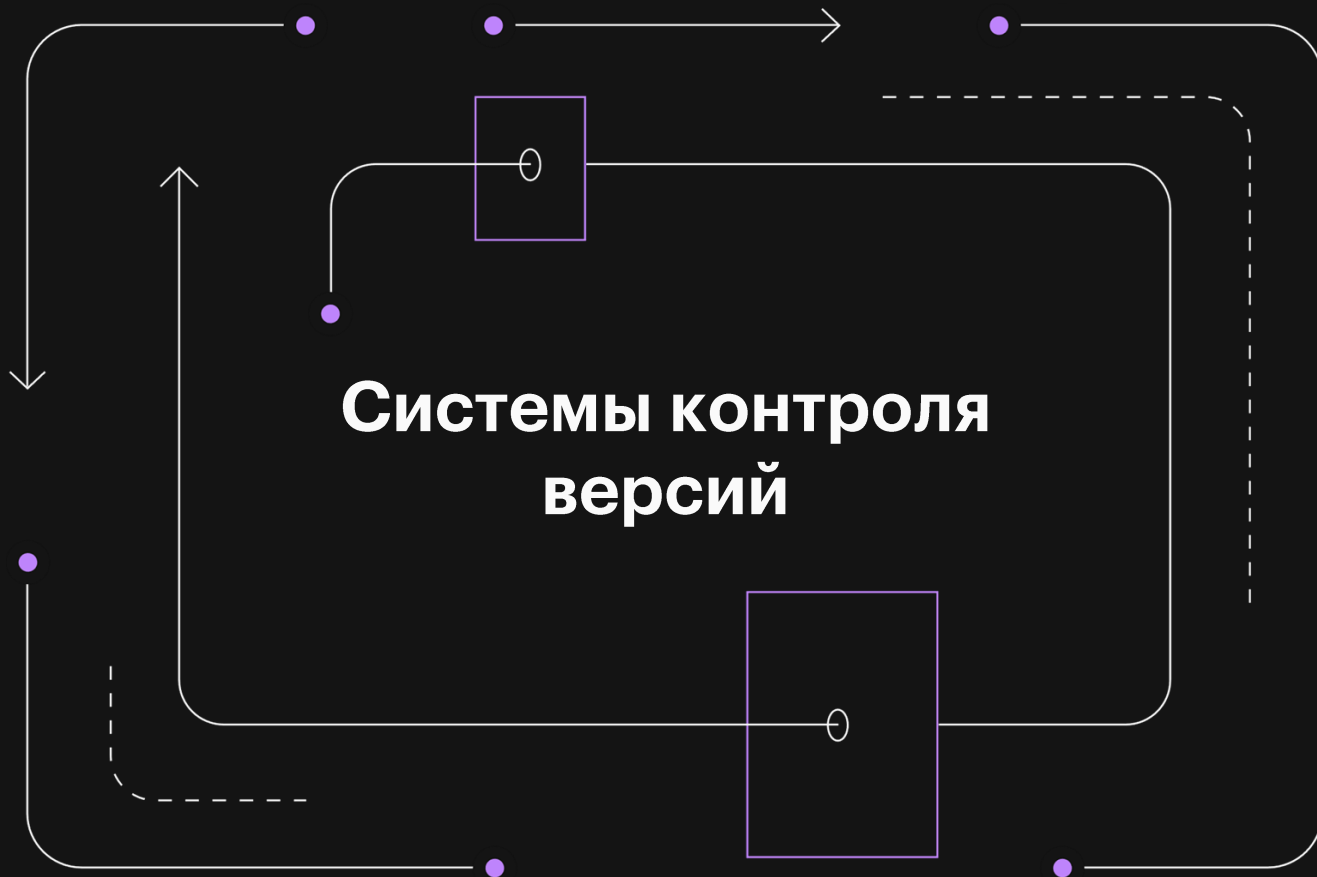
Узнаем, как организовать работу в команде с помощью GitLab — настройка репозитория, работа с SSH-ключами



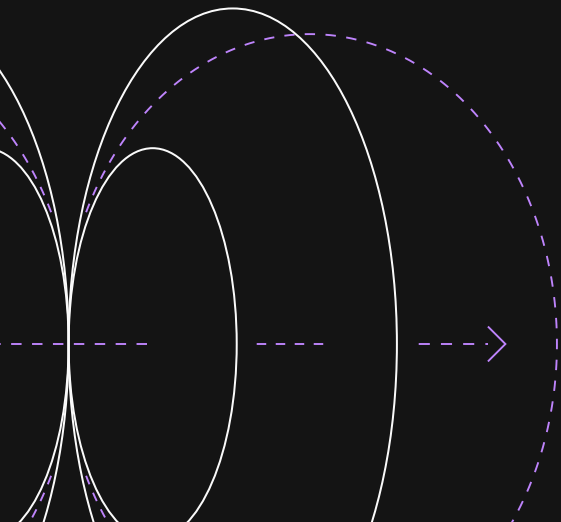
Узнаем, как правильно отправлять код, создавать Merge Requests и участвовать в код-ревью



Посмотрим, как избежать типичных ошибок при работе с Git и GitLab



Почему без Git код превращается в хаос



Представь, что ты разрабатываешь приложение. Ты написал код, протестировал его и сохранил в файле. Через неделю обнаруживаешь, что программа больше не работает, но не помнишь, какие изменения привели к ошибке.

Если восстановить старую версию вручную, то сразу возникают вопросы.

- Где хранить разные версии файлов?
- Как понять, какая версия правильная?
- Что делать, если в команде у каждого разработчика свои изменения в коде?

Без системы контроля версий код становится неуправляемым. Именно поэтому Git — ключевой инструмент в современной разработке.

Git в командной разработке



Ранее ты писал код и запускал его в одиночку, но в реальной работе программирование — это процесс, где изменения в коде происходят постоянно. Git помогает разработчикам:

- Хранить историю изменений, чтобы не потерять важные правки.
- Работать в команде, не мешая друг другу.
- Откатываться к рабочим версиям, если что-то пошло не так.

Git — контроль версий и независимая работа

Git — это распределённая система контроля версий (VCS), которая отслеживает изменения в файлах, позволяет управлять версиями кода, вести разработку в разных ветках и объединять изменения от нескольких разработчиков.

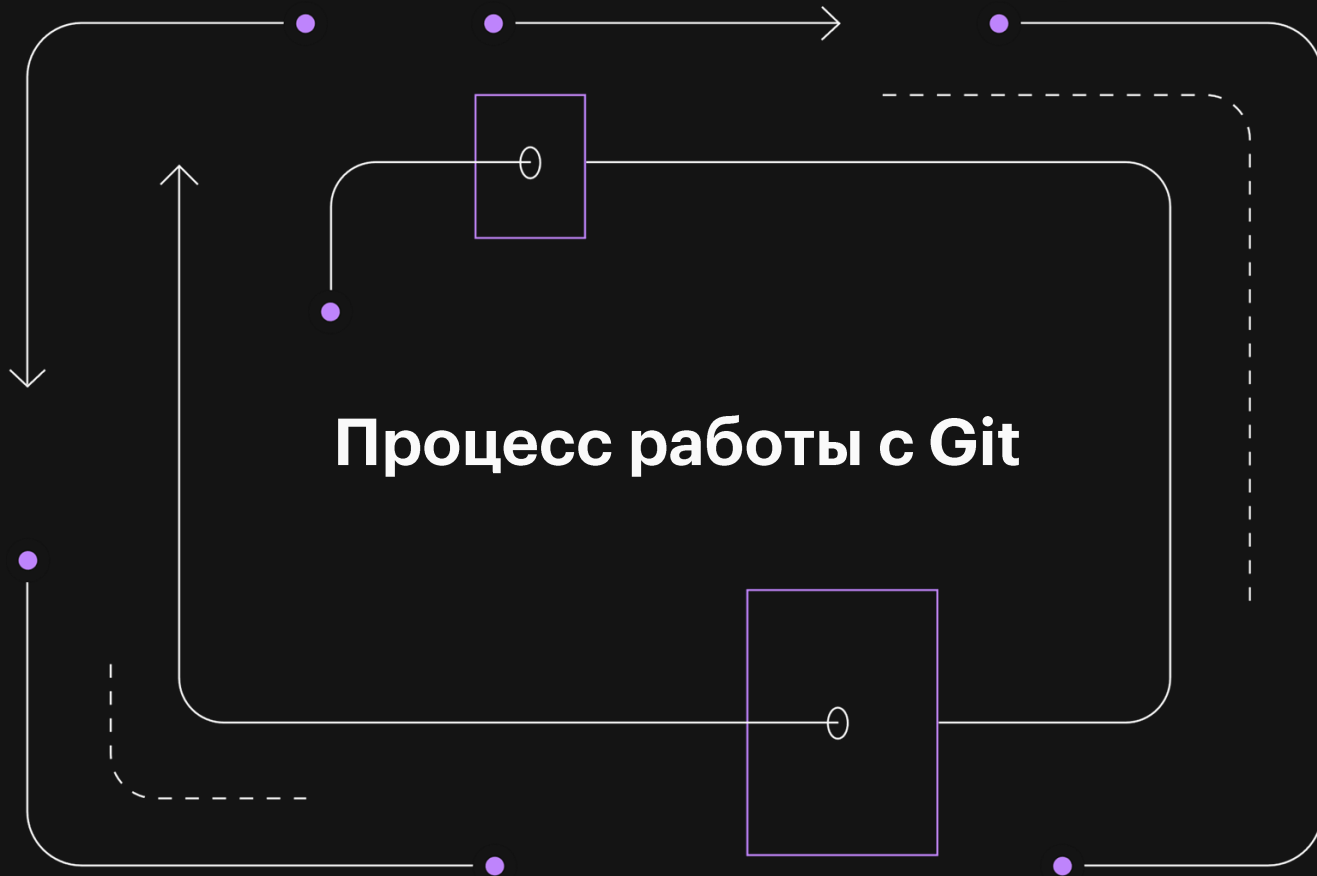


- Git позволяет сохранять каждое изменение кода в виде зафиксированных точек (коммитов) и при необходимости возвращаться к любой версии проекта.
- Каждый разработчик может создавать свою ветку — копию основного кода, в которой может работать независимо.
- Git не требует центрального сервера, каждый разработчик имеет локальную копию репозитория.
- Каждое изменение кодируется и подписывается, что предотвращает потерю данных или их подмену.

Установка Git



Git необходимо установить, он есть не у всех систем «из коробки».
Чтобы найти инструкцию по установке — загляни на welcome-страницу курса.



Инициализация и структура репозитория в Git



Где живет история проекта в Git



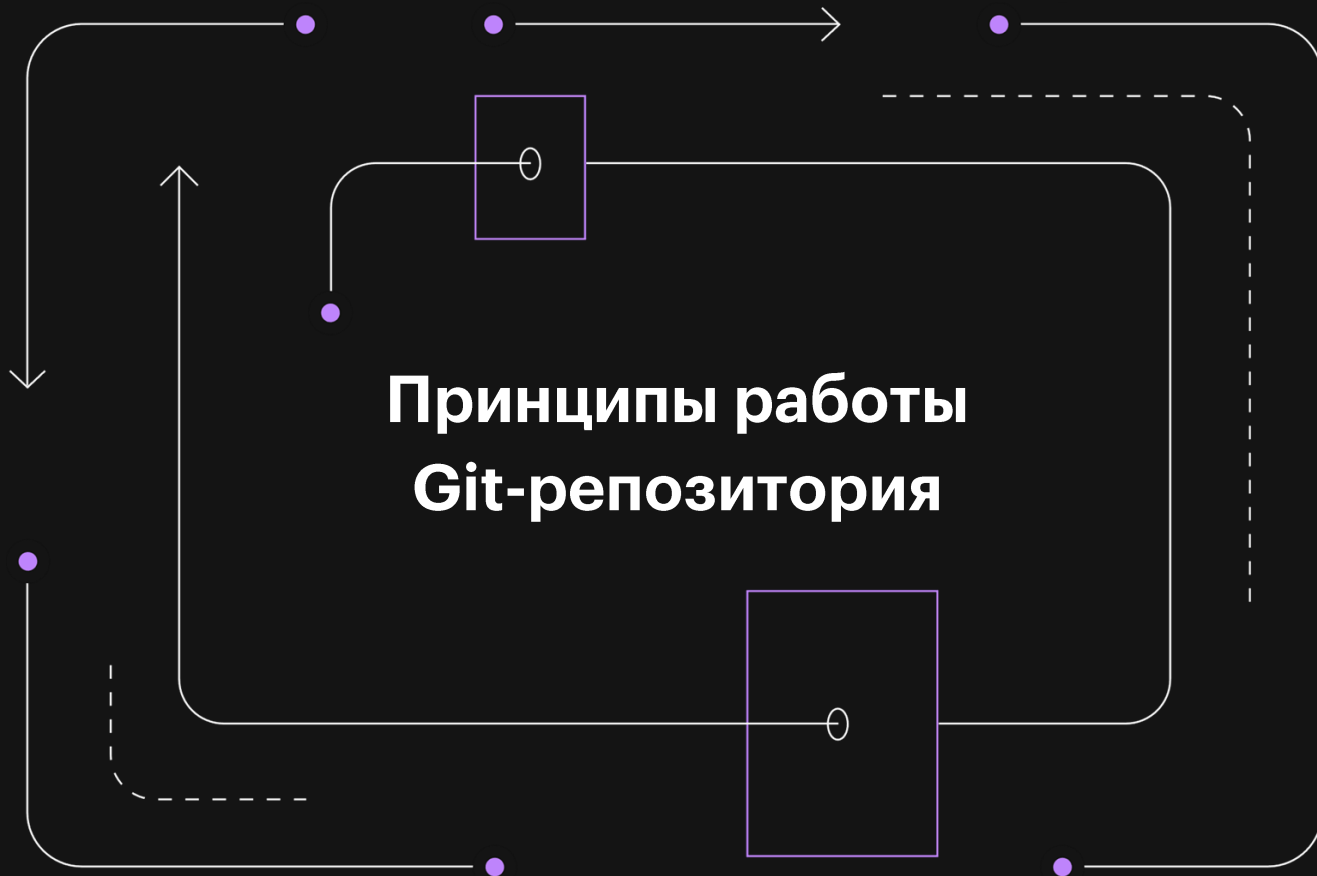
Ты начал разрабатывать приложение и хочешь вести историю его изменений. Где и как Git будет хранить эту информацию?



Репозиторий Git — это хранилище, где хранятся все изменения проекта, коммиты, ветки и файлы. Он может быть локальным (на твоём компьютере) и удалённым (на сервере, например, в GitLab).



Инициализация репозитория — это процесс создания структуры Git внутри проекта, позволяющий Git отслеживать изменения в файлах.



Принципы работы Git-репозитория



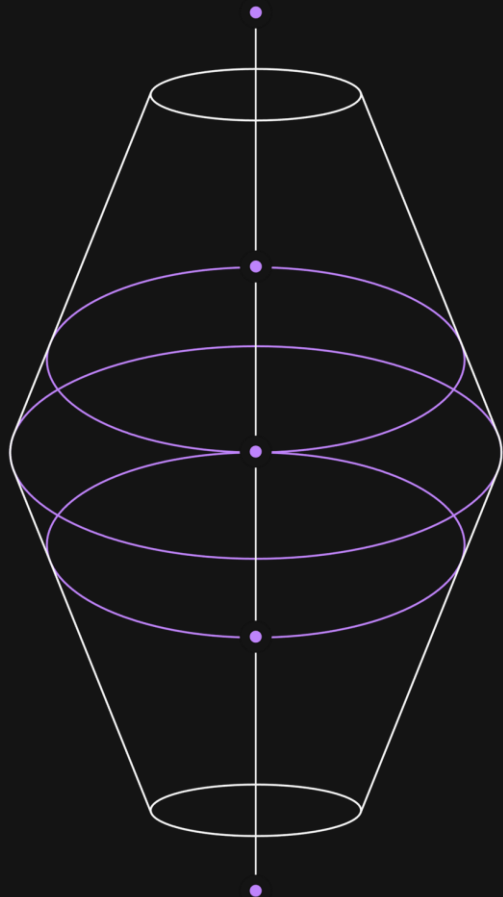
Репозиторий хранит всю информацию в скрытой папке `.git`, которая создается в корневой директории проекта.

```
[example@central central] ls .git
HEAD          description   info          refs
config        hooks        objects
```

Принципы работы Git-репозитория

В Git файлы могут находиться в одном из трёх состояний:

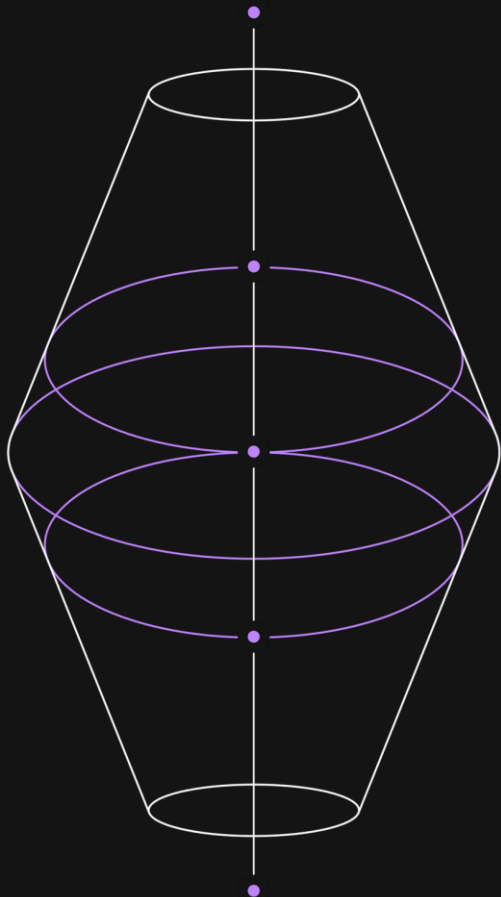
- **Untracked** (неотслеживаемые)
- **Staged** (подготовленные)
- **Committed** (зафиксированные)



Принципы работы Git-репозитория

Структура хранения данных

В отличие от обычных копий файлов, Git хранит разницу (diff) между версиями, что позволяет эффективно управлять изменениями.



```
example@central central git add text.txt
example@central central git commit -m "init"
[main (root-commit) 28db434] init
 1 file changed, 1 insertion(+)
 create mode 100644 text.txt
example@central central nano text.txt
example@central central git diff
diff --git a/text.txt b/text.txt
index 58c9bdf..c200906 100644
--- a/text.txt
+++ b/text.txt
@@ -1,1 +1,2 @@
-111
+222
```

Инициализация Git-репозитория



1. Создание проекта

Разработчик создаёт папку для своего проекта и хочет, чтобы Git начал отслеживать изменения.

2. Инициализация репозитория

Разработчик инициализирует репозиторий

```
git init
```

Что внутри папки .git

Git создаёт в папке проекта скрытую директорию .git, в которой хранятся:

HEAD — указатель на текущую ветку

objects/ — файлы коммитов,
истории и изменений

config — настройки репозитория

refs/ — информация о ветках

Инициализация Git-репозитория



3. Добавление файлов

Git пока не отслеживает файлы, но их можно добавить в область подготовки.

Область подготовки (Staging Area) — это промежуточное хранилище, в котором находятся файлы перед окончательной фиксацией в истории изменений.

4. Фиксация изменений

После фиксации изменений они становятся частью истории проекта.

Графическое представление структуры Git-репозитория

Проект/

```

| — .git/      # Скрытая папка с данными репозитория
|   | — HEAD   # Указатель на текущую ветку
|   | — config  # Конфигурационные файлы Git
|   | — objects/ # Хранилище всех изменений
|   | — refs/   # Ссылки на ветки и коммиты
| — main.py     # Файл проекта
| — README.md   # Описание проекта
  
```

Коммиты и история изменений в Git



Представь, ты начал писать код, изменил несколько файлов, сохранил их в редакторе, но Git ничего о них не знает. Если сейчас закрыть проект или допустить ошибку, нет возможности откатиться назад.

Git решает эту проблему с помощью фиксации изменений — коммитов.

Коммиты и история изменений в Git



Каждый раз, когда ты фиксируешь изменения, Git добавляет их в историю, позволяя тебе в любой момент:

- вернуться к предыдущей версии кода;
- посмотреть, кто и когда внёс изменения;
- работать с разными версиями одновременно, не мешая другим разработчикам.

Коммиты и история изменений в Git

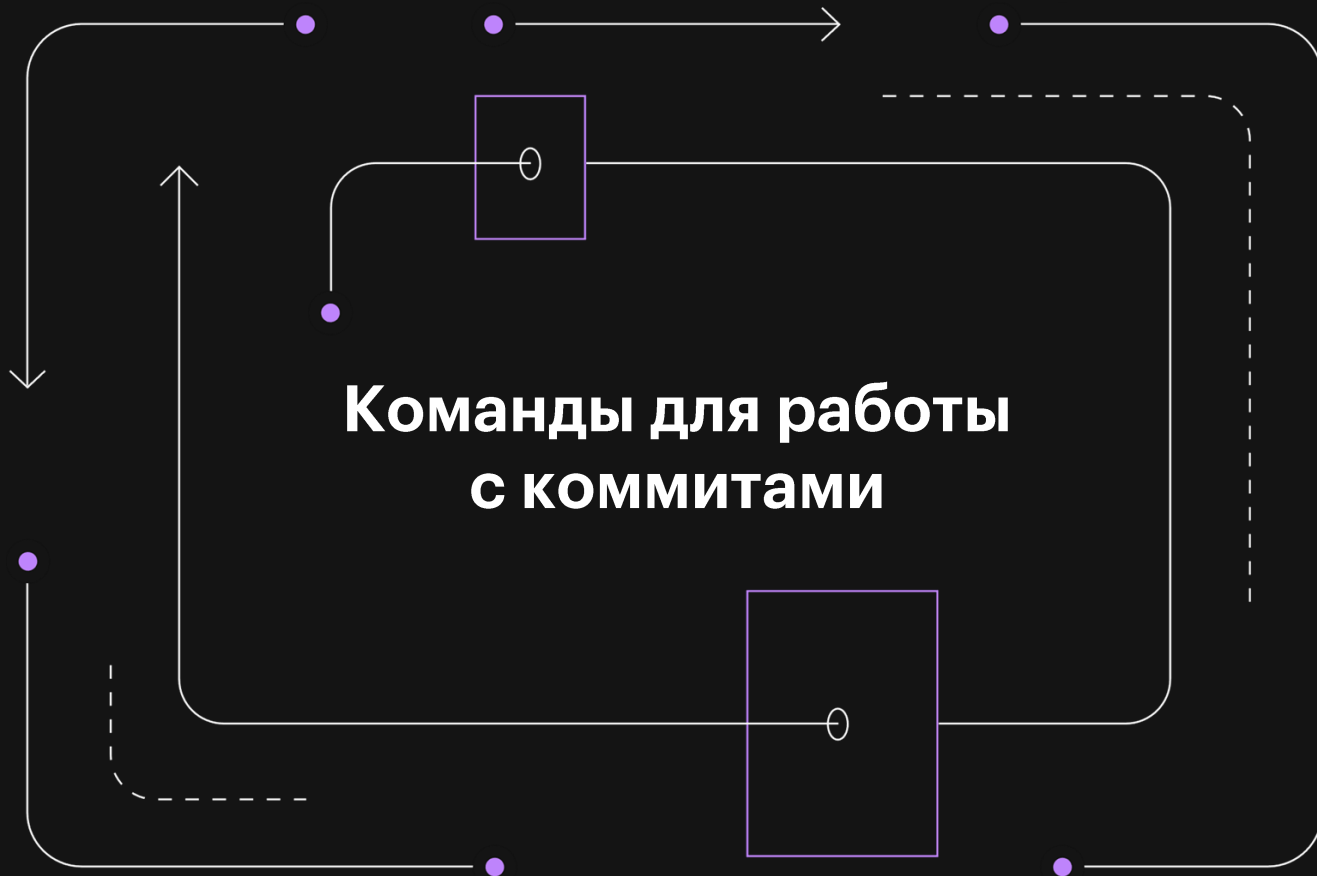


Коммит (commit) — это зафиксированное изменение в репозитории Git. Каждый коммит содержит:

- уникальный идентификатор (hash) — строку, по которой можно найти коммит;
- сообщение (commit message) — пояснение, что изменилось;
- авторство — имя и email разработчика;
- временную метку — дату и время коммита.

История изменений (commit history) — это последовательность коммитов, отражающая все изменения в проекте с момента его создания.

Команды для работы с коммитами



Добавление файла в область подготовки



Перед фиксацией изменений файлы нужно добавить в Staging Area:

```
git add <имя_файла>
```

или

```
git add.
```

Эта команда добавляет **все изменённые файлы** в область подготовки.

Создание коммита



Чтобы управлять коммитами, в Git есть специальные команды:

```
git commit -m 'Добавлено описание изменений'
```

Фиксирует изменения в репозитории с сообщением.

Если пропустить `-m`, откроется текстовый редактор, где можно написать сообщение.

Просмотр истории коммитов



Существует команда, которая выведет список всех коммитов в репозитории с их идентификаторами, авторами и датами (можно посмотреть краткую историю):

```
git log
```

Выведет список всех коммитов в репозитории с их идентификаторами, авторами и датами.

Чтобы увидеть краткую историю:

```
git log --oneline
```

Коммиты будут отображены одной строкой.

Управление коммитами



4. Просмотр изменений перед коммитом

```
git status
```

Показывает состояние файлов — какие изменены, добавлены или не отслеживаются.

```
git diff
```

Показывает, какие изменения были внесены в файлы.

Просмотр деталей коммита в Git



Чтобы управлять коммитами, в Git есть специальные команды.

5. Просмотр конкретного коммита:

```
git show <ID коммита>
```

Выведет полную информацию о коммите, включая внесенные изменения.

Просмотр деталей коммита в Git



6. Откат к предыдущему коммиту

Иногда нужно вернуть проект к предыдущему состоянию:

```
git checkout <ID коммита>
```

Это временный переход, можно вернуться обратно в последнюю версию командой:

```
git checkout main
```

Просмотр деталей коммита в Git



7. Откат к предыдущему коммиту

Если нужно навсегда отменить изменения, можно использовать:

```
git reset --hard <ID коммита>
```

Важно. Эта команда удаляет все изменения, сделанные после указанного коммита. Никогда не используй флаг `--hard` в рабочих проектах!

Работа с коммитами и историей изменений

ПРИМЕР



1. Добавление новых файлов
2. Проверка состояния репозитория
3. Добавление файла в область подготовки
4. Создание первого коммита
5. Изменение файла и создание нового коммита
6. Просмотр истории изменений

Графическое представление истории изменений



Начало проекта

|

[Коммит A] - "Создан основной файл"

|

[Коммит B] - "Добавлена функция X"

|

[Коммит C] - "Исправлен баг в функции X"

|

[Текущий коммит] - "Добавлен новый модуль"

Файл .gitignore в Git



Файл .gitignore в Git

В проекте есть файлы, которые **не должны попадать в Git**:

- временные файлы IDE (например, `.vscode/` или `.idea/`);
- виртуальные окружения (`.venv`, `venv` или `env`);
- секретные данные, например, файлы с паролями (`config.env`, `.env`).
- файлы, создаваемые системой (`.DS_Store`, `Thumbs.db`).

Если их случайно закоммитить, это может привести к проблемам:

- Лишние файлы засоряют репозиторий.
- Пароли или токены могут попасть в общий доступ.
- Проект может работать по-разному на разных компьютерах.

Файл `.gitignore` в Git

Чтобы избежать таких проблем, Git использует файл `.gitignore`, в котором указывается список файлов и папок, которые Git должен игнорировать.

`.gitignore` — это текстовый файл, в котором перечислены файлы и директории, которые Git должен игнорировать (не отслеживать и не включать в коммиты).

Как работает .gitignore



`.gitignore` — фильтр для Git

В корне проекта — список файлов и папок, которые не отслеживаются Git (Создай файл в корневой папке проекта).

Работает только для неотслеживаемых файлов — уже закоммиченные файлы надо удалить из истории отдельно.

Файл .gitignore в Git

3. Можно использовать шаблоны

- *.log – игнорировать все файлы с расширением .log.
- logs/ – игнорировать всю папку logs.
- !config.example.json – не игнорировать config.example.json, даже если config.json в .gitignore.

Совет:

- Для Python-проектов есть готовые шаблоны .gitignore
- Если используешь JetBrains IDE — раскомментируй .idea/

Пример использования .gitignore

ПРИМЕР



Для начала необходимо создать файл .gitignore и добавить в него файлы/папки, которые необходимо игнорировать.

Если файл уже был закоммичен и его нужно удалить из истории:

```
git rm --cached config.env
```





Ветвление в Git — работа над функциями без конфликтов

- Одна ветка — один набор файлов: до этого мы фиксировали изменения только в основной ветке.
- Нужна новая функция? Создай отдельную ветку, чтобы не ломать рабочую версию кода.
- Командная работа: каждый разработчик ведет свою ветку — меньше шансов на конфликты.
- Объединение изменений: ветки можно безопасно сливать в основную версию.

Ветвление в Git — работа над функциями без конфликтов

- ➔ Ветка (branch) — это независимая линия разработки, которая позволяет вносить изменения в код без влияния на основную версию проекта.
- ➔ Слияние (merge) — это объединение изменений из одной ветки в другую.
- ➔ Конфликт слияния (merge conflict) — ситуация, когда два разработчика изменили одну и ту же часть файла, и Git не может автоматически объединить их версии.

Как работают ветки в Git?



Ветка main (или master)



По умолчанию у каждого репозитория есть основная ветка (main или master).



Раньше стандартом наименования веток был master, но с недавних пор главная ветка проекта должна называться main.

Как работают ветки в Git?



2. Создание новых веток

Разработчик может создать новую ветку и работать в ней, не затрагивая основную версию кода.

3. Переключение между ветками

Можно переключаться между разными ветками и работать с разными версиями кода.

Как работают ветки в Git?



4. Слияние изменений

Когда работа завершена, изменения из новой ветки можно объединить с основной.

5. Разрешение конфликтов

Если два человека изменили одну и ту же строку кода, Git попросит вручную выбрать, какая версия остаётся.

Разбор команд работы с ветками



Как и для коммитов, в Git для работы с ветками существует множество команд:

1. Просмотр списка веток

```
git branch
```



Выведет список всех локальных веток.



Разбор команд работы с ветками



2. Создание новой ветки

```
git branch feature/new-function
```



Создаёт новую ветку feature/new-function



Разбор команд работы с ветками



3. Переключение на новую ветку

```
git checkout feature/new-function
```

или (более новый вариант команды)

```
git switch feature/new-function
```



Теперь все изменения кода будут происходить в новой ветке.



Разбор команд работы с ветками



В версии Git 2.23 было принято решение разделить функциональность работы с ветками, поэтому для операции переключения между ветками была добавлена команда `git switch`.

Разбор команд работы с ветками



4. Создание ветки и переключение в нее одной командой

```
git checkout -b feature/new-function
```

или

```
git switch -c feature/new-function
```



Создает новую ветку и сразу переключается в неё.



Разбор команд работы с ветками



5. Слияние ветки с main

Когда работа в ветке завершена, изменения можно объединить с основной веткой:

```
git checkout main  
git merge feature/new-function
```

или

```
git switch main  
git merge feature/new-function
```



Git объединит изменения из feature/new-function в main



Разбор команд работы с ветками



6. Удаление ветки после слияния

Если ветка больше не нужна, ее можно удалить:

```
git branch -d feature/new-function
```



Работа с ветками и слиянием

ПРИМЕР



1. Создание новой ветки
2. Изменения в коде
3. Завершение работы и слияние
5. Удаление ветки





Конфликты при слиянии в Git



Если два человека изменили одну и ту же строку кода, Git не сможет автоматически объединить файлы.

Основные шаги для решения конфликта:

1. Git останавливает слияние и сообщает о конфликте (CONFLICT в git status).
2. Открыть файл с конфликтом — Git отметит конфликтные блоки с помощью маркеров.

Конфликты при слиянии в Git



3. Вручную выбрать правильный вариант:

- оставить один из вариантов или объединить оба;
- удалить маркеры (<<<<<<, =====, >>>>>>).

4. Добавить исправленный файл в индекс:

```
git add <файл>
```

5. Завершить слияние коммитом:

```
git commit
```

Конфликт после git merge feature-branch

ПРИМЕР

CONFLICT (content): Merge conflict in app.py
Automatic merge failed; fix conflicts and
then commit the result.



Графическое представление работы с ветками



[Коммит А] - "Создан основной файл" (main)

|

[Коммит В] - "Добавлена функция X" (main)

|

├── [Коммит С] - "Начата работа над Y" (feature/new-function)

|

├── [Коммит D] - "Продолжена работа над Y" (feature/new-function)

|

└── [Коммит E] - "Функция Y завершена" (feature/new-function)

|

[Слияние] - "Объединение feature/new-function в main"

Работа с удаленными репозиториями в GitLab



Удаленные репозитории и совместная работа

До этого момента мы работали с Git локально — сохраняли изменения в репозитории на своём компьютере.

Но в реальной разработке код пишется в команде, и его нужно хранить в общем месте, доступном для всех разработчиков.

GitLab решает эту задачу.

Удаленные репозитории и совместная работа

GitLab позволяет:

- Хранить код на удаленном сервере.
- Совместно разрабатывать проект.
- Проводить код-ревью и управлять изменениями.
- Автоматизировать тестирование и развертывание.

Удаленные репозитории и совместная работа

- Удаленный репозиторий (remote repository) — это репозиторий, который хранится не на локальном компьютере, а на сервере.
- GitLab — это веб-платформа для хранения Git-репозиториях, управления разработкой, код-ревью и CI/CD.
- Push — отправка локальных изменений в удалённый репозиторий.
- Pull — загрузка актуальных изменений из удалённого репозитория в локальный.

Как работать с удаленными репозиториями?



Новый проект в GitLab



1. Создание проекта в GitLab

Код должен быть загружен в удаленный репозиторий, чтобы команда могла работать с ним.

2. Настройка SSH-доступа

Чтобы не вводить пароль каждый раз (если пароль вообще можно использовать для аутентификации в компании) и подключение к репозиториям было безопасным, можно настроить SSH-ключ для взаимодействия с GitLab.

Новый проект в GitLab



3. Связывание локального репозитория с удалённым

Git позволяет работать с удалёнными репозиториями, добавляя их как "remote".

4. Отправка и получение изменений

Команда `git push` отправляет изменения в удалённый репозиторий, а `git pull` загружает актуальную версию кода.

Разбор команд для работы с удаленными репозиториями



Команды Git для работы с удаленными репозиториями

Для работы с удаленными репозиториями есть команды, давай с ними познакомимся:

1. Добавление удаленного репозитория

Если локальный репозиторий уже существует, но не привязан к удаленному его можно привязать к репозиторию в GitLab командой:

```
git remote add origin URL-репозитория
```

Команды Git для работы с удаленными репозиториями

Здесь `origin` — стандартное имя удалённого репозитория. Имя может быть любым, `origin` это общепринятое

Указывай тот тип URL – SSH или HTTPS, который подходит под твой настроенный безопасный способ подключения к GitLab. Далее в примерах мы будем использовать SSH.

Если мы забыли выполнить это действие git напомним нам об этом и предложит выполнить команду для связывания локального и удаленного репозитория

Команды Git для работы с удаленными репозиториями

2. Проверка подключенных удалённых репозиториев

```
git remote -v
```



Показывает список всех удалённых репозиториев, привязанных к проекту.

Команды Git для работы с удаленными репозиториями

3. Отправка изменений в удаленный репозиторий

```
git push origin main
```

Отправляет изменения из локальной ветки main в удаленный репозиторий origin.
Если работаешь с новой веткой:

```
git push --set-upstream origin feature/new-function
```



Эта команда создаст новую ветку на сервере и привяжет её к локальной.

Команды Git для работы с удаленными репозиториями

4. Получение актуальных изменений

```
git pull origin main
```



Загружает свежие изменения из удалённого репозитория и объединяет их с локальной веткой.

Команды Git для работы с удаленными репозиториями

5. Клонирование удаленного репозитория

Если нужно скачать проект с GitLab на свой компьютер:

```
git clone <URL-репозитория>
```



Создаст локальную копию репозитория.

Пример отправки локального проекта в GitLab

ПРИМЕР

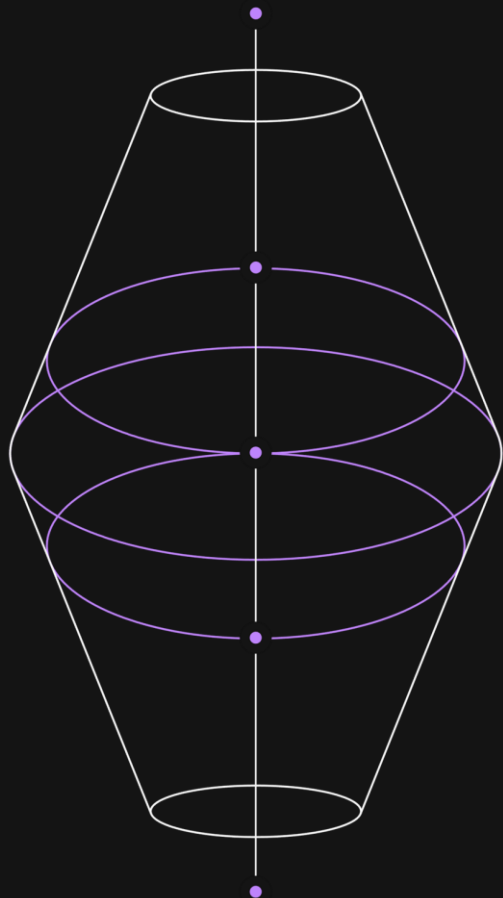


- Создание репозитория в GitLab
- Привязка локального репозитория к GitLab
- Совместная работа с кодом



Работа с Merge Requests и код-ревью в GitLab





Работа с Merge Requests и код-ревью в GitLab



Мы уже знаем, как подключаться к удаленному репозиторию, работать с ветками и отправлять код в GitLab.

Но как организовать процесс командной разработки?

Если каждый будет напрямую вносить изменения в основную ветку (main), это может привести к ошибкам и конфликтам.

GitLab предлагает решение — Merge Requests (MR).

Работа с Merge Requests и код-ревью в GitLab



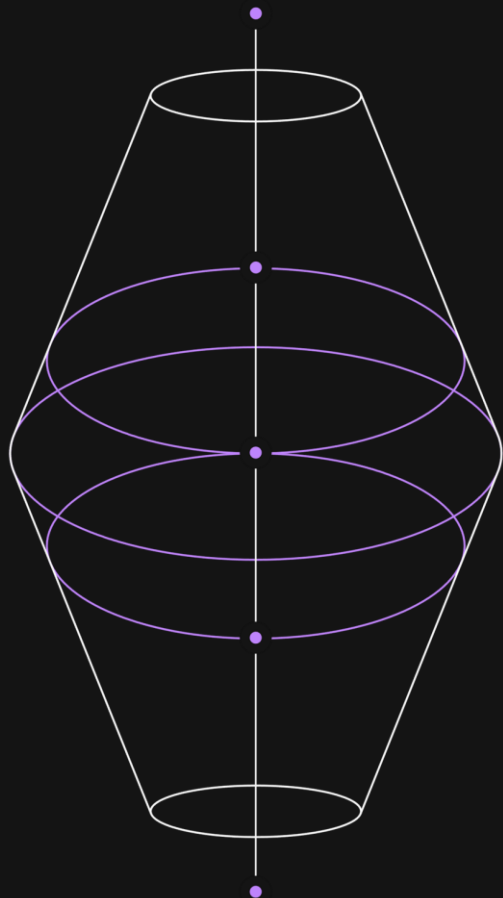
Merge Request (MR) — это механизм в GitLab, который позволяет объединять изменения из одной ветки в другую после код-ревью



Код-ревью (Code Review) — процесс проверки кода коллегами перед его включением в основной проект



Approvals (Подтверждения) — настройка в GitLab, требующая одобрения нескольких разработчиков перед слиянием



Работа с Merge Requests и код-ревью в GitLab



Процесс работы с Merge Requests

1. Разработчик создаёт новую ветку
2. Создаётся Merge Request (MR)
3. Код-ревью
4. Исправление замечаний
5. Подтверждение и слияние

Работа с Merge Requests и код-ревью в GitLab



Перейди в Merge Requests → New Merge Request



Выбери свою ветку (feature/new-function) и основную (main)



Напиши заголовок и описание изменений



Нажми Create Merge Request

Работа с Merge Requests и код-ревью в GitLab

Как работать с Merge Requests



Коллеги просматривают изменения и оставляют комментарии (проводят код-ревью).



Если нужно внести правки, разработчик делает новые коммиты и отправляет их в ту же ветку:

```
git add .  
git commit -m 'Исправления после ревью'  
git push origin feature/new-function
```

Работа с Merge Requests и код-ревью в GitLab

Как работать с Merge Requests



GitLab автоматически обновит Merge Request.



Когда код одобрен, его можно объединить с main.



В GitLab нажми Merge или сделай это вручную с помощью команд:

```
git checkout main
git merge feature/new-function
git push origin main
```

Работа с Merge Requests и код-ревью в GitLab

Как работать с Merge Requests



После слияния ветка `feature/new-function` больше не нужна, ее можно удалить:

```
git branch -d feature/new-function  
git push origin --delete feature/new-function
```



Что делать, если возник конфликт?



Если изменения из feature/new-function пересекаются с main, Git не сможет автоматически выполнить слияние.

В этом случае:

1. Перейди в GitLab, найди файл с конфликтом.
2. Выбери нужную версию кода или внеси правки вручную.
3. Закоммить исправленный файл.



Пример процесса Merge Request в GitLab

[Разработчик создал ветку feature/login]
|
[Разработчик внёс изменения и отправил их в GitLab]
|
[Создан Merge Request → Код отправлен на ревью]
|
[Коллеги оставили замечания → Разработчик внёс исправления]
|
[MR одобрен → Изменения слиты в main]
|
[Ветка feature/login удалена]

Частые ошибки при работе с Git или GitLab



Частые ошибки при работе с Git или GitLab

Работая с Git и GitLab, разработчики часто сталкиваются с ошибками:

- Неудачные слияния, которые ломают код.
- Потеря коммитов при `git reset`.
- Проблемы с аутентификацией при `git push`

Эти ошибки могут привести к потере данных или долгому поиску решения.

Поэтому важно заранее знать, как их избежать и как исправить, если они уже случились.

Ошибка 1: Закоммитили файл, который должен быть в .gitignore

Ситуация

Ты случайно добавил в коммит файл `.env`, который должен был игнорироваться.

Решение

1. Добавь файл в `.gitignore`
`.env`

Ошибка 1: Закоммитили файл, который должен быть в .gitignore

Решение

2. Удали его из индекса Git, не трогая локальную копию

```
git rm --cached config.env
```

Закоммить изменения:

```
git commit -m 'Удалён из репозитория файл .env'  
git push origin main
```

Теперь файл останется в локальной папке, но Git его игнорирует.

Ошибка 2: Конфликт при слиянии веток

Ситуация

Ты пытаешься выполнить `git merge`, но Git сообщает о конфликте.

Решение

1. Открой файл с конфликтом (Git покажет, где возникли изменения).
2. Вручную выбери нужную версию кода, удалив лишние строки (включая служебные указатели):

Ошибка 2: Конфликт при слиянии веток

```
<<<<<< HEAD
код из основной ветки
=====
код из сливаемой ветки
>>>>>> feature/new-function
```

После исправления закоммитить изменения

```
git add .
git commit -m 'Исправлен конфликт слияния'
```

Ошибка 3: git push требует аутентификацию при каждом запуске

Ситуация

При каждом git push тебе нужно вводить логин и пароль.

Решение

Лучше использовать SSH-ключи.

Для этого необходимо изменить URL репозитория и использовать командой:

```
git remote set-url origin  
git@git.culab.ru:username/repo.git
```


Ошибка 3: git push требует аутентификацию при каждом запуске

Если ты работаешь через HTTPS, можно включить сохранение пароля:

```
git config --global credential.helper store
```

Теперь Git запомнит твои учетные данные.

Ошибка 4: fatal: remote origin already exists при добавлении удалённого репозитория



Ситуация

Ты попытался выполнить

```
git remote add origin <URL-репозитория>
```

но получил ошибку, потому что удаленный репозиторий уже настроен.

Ошибка 4: fatal: remote origin already exists при добавлении удалённого репозитория



Решение

Проверить существующие remote:

```
git remote -v
```

Если origin уже есть, можно его заменить:

```
git remote set-url origin <новый-URL>
```

Ошибка 5: fatal: refusing to merge unrelated histories

Ситуация

Ты пытаешься выполнить `git pull`, но получаешь ошибку:

```
fatal: refusing to merge unrelated histories
```

Это случается, если у локального и удалённого репозитория нет общей истории.

Решение

Разрешить слияние, добавив флаг `--allow-unrelated-histories`:

```
git pull origin main --allow-unrelated-histories
```

Ошибка 6: git push отклонен из-за несоответствия истории

Ситуация

Ты пытаешься отправить изменения в удалённый репозиторий, но получаешь ошибку:

```
fatal: rejected push to origin/main
```

Это происходит, если на сервере уже есть изменения, которых нет в твоей локальной копии.

Решение

Сначала подтяни изменения с сервера:

```
git pull origin main --rebase
```

Ошибка 6: git push отклонен из-за несоответствия истории

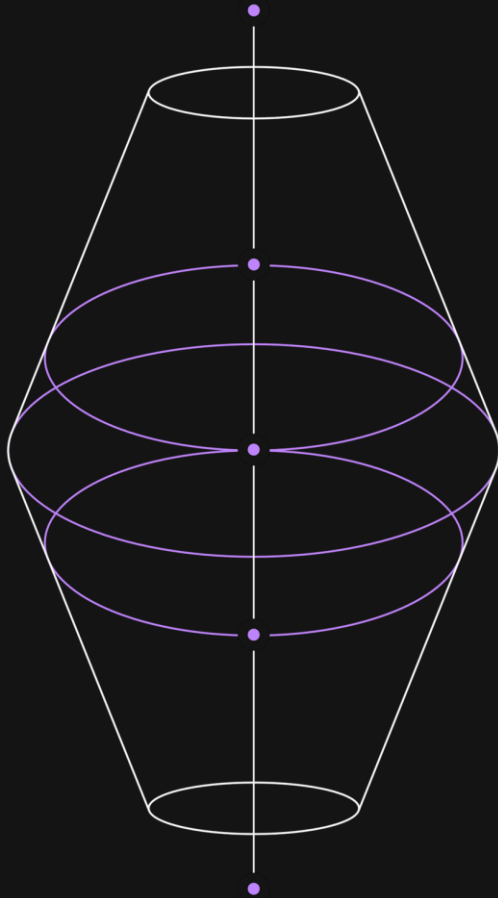
Затем снова отправь код:

```
git push origin main
```

Советы как избежать ошибок



Советы как избежать ошибок



1. Перед git push всегда выполняй git pull
2. Перед git merge всегда выполняй git status
3. Пиши осмысленные коммиты
4. Добавляй файлы в .gitignore заранее

Лучшие практики оформления Git-репозитория и README



Зачем важна структура репозитория?

Когда ты создаешь репозиторий, важно не просто загружать код, но и делать его понятным для других разработчиков (и для себя в будущем).

- Хорошо организованный репозиторий:
- Позволяет быстро разобраться в проекте.
- Делает документацию доступной и понятной.
- Облегчает работу в команде.

Структура репозитория

Обычно репозиторий может выглядеть примерно так:

```

my_project/      # Корневая директория проекта
├── src/          # Исходный код
│   ├── main.py   # Основной скрипт
│   └── utils.py  # Вспомогательные функции
├── tests/        # Тесты
│   ├── test_main.py # Тестирование main.py
│   └── test_utils.py # Тестирование utils.py
├── docs/         # Документация проекта
│   ├── installation.md # Инструкция по установке
│   ├── usage.md      # Примеры использования
│   └── api.md        # Описание API
├── .gitignore    # Исключает файлы из репозитория
├── requirements.txt # Список зависимостей
├── README.md     # Главный файл документации
└── LICENSE       # Лицензия проекта
    
```

Структура репозитория



На разных языках программирования, при использовании разных фреймворков структура репозитория может меняться, но обычно она всегда содержит несколько ключевых элементов — исходный код, тесты, документацию, лицензию.

На следующих неделях мы будем подробно знакомиться со структурой проектов.

README: зачем он нужен?



На текущем этапе сделаем акцент на README файле.

README.md — это основной файл документации проекта.

Если кто-то открывает твой репозиторий, первым делом он видит README. Без него проект может быть непонятен даже его автору через несколько месяцев.

README должен отвечать на вопросы:

- Что это за проект?
- Как его установить?
- Как его использовать?
- Какие есть зависимости?
- Как вносить вклад в проект?

Структура хорошего README

Стандартный README
должен включать:



Название проекта

Краткое описание: что делает этот проект и кому он полезен.

Установка

Инструкции по установке:

```
git clone https://gitlab.com/username/project.git
```

```
cd project
```

```
pip install -r requirements.txt
```

Использование

Примеры запуска и команд:

```
from my_project import main
```

```
main.run()
```

Структура проекта

Описание ключевых файлов и директорий.

Вклад в проект

Как внести изменения, создать issue или merge request.

Лицензия

Информация о лицензии проекта.

Лучшие практики по оформлению README

1. Используйте заголовки и форматирование

– заголовок

– подзаголовок

****жирный текст****, *_курсив_*

Блоки кода оформляются в тройных кавычках:

```
```bash
pip install -r requirements.txt
```
```

Лучшие практики по оформлению README

2. Добавляй скриншоты или GIF-анимации

Визуальные элементы помогают быстрее понять проект.

![Пример работы](https://example.com/screenshot.png)

Лучшие практики по оформлению README

3. Добавляй бейджи статусов

Бейджи позволяют показать версию, статус тестов и другие метрики.

```
![CI Status](https://img.shields.io/badge/build-passing-green)
```

Лучшие практики по оформлению README

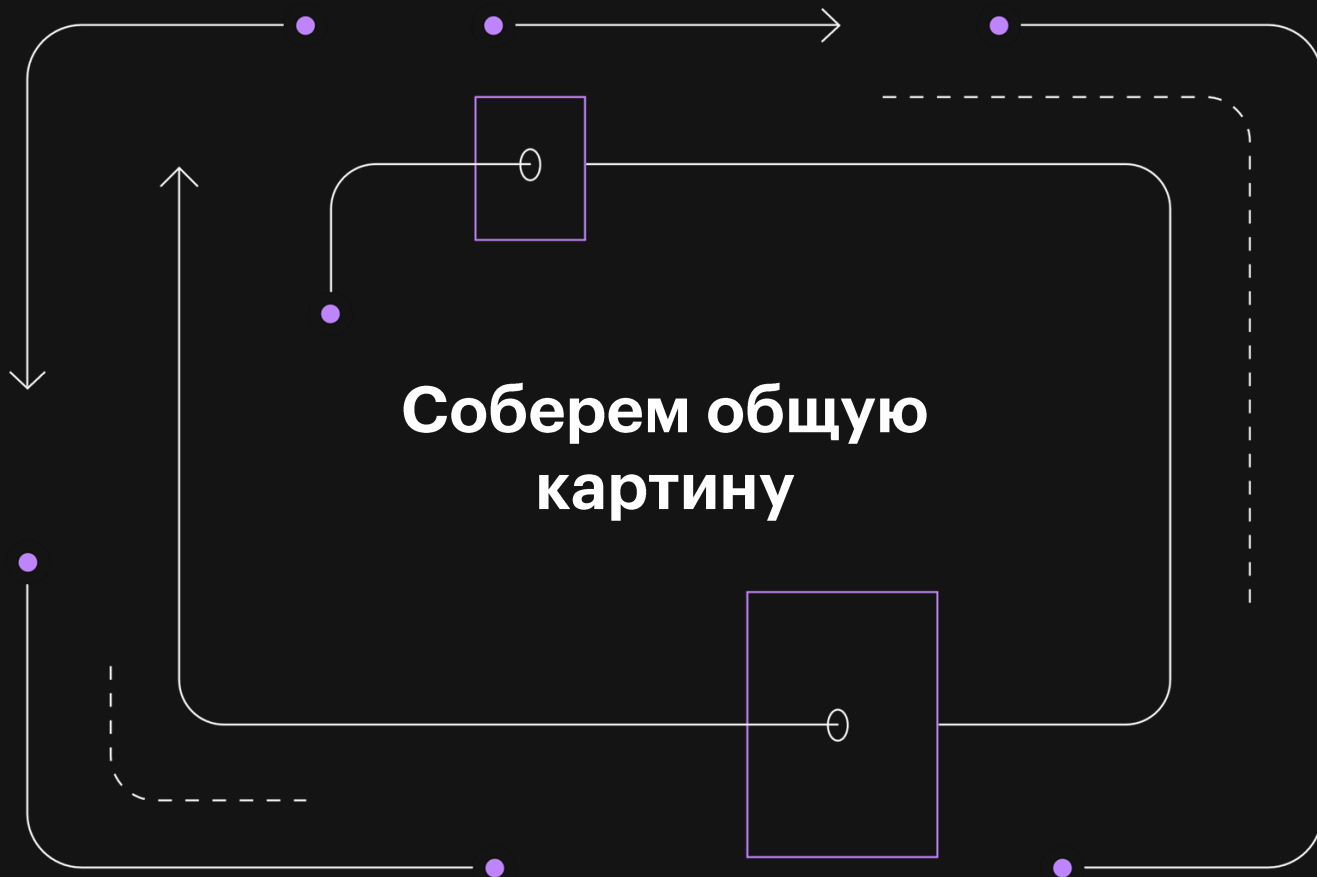
4. Используй таблицы для удобства

| Файл | Описание |
|-------------|------------------------|
| main.py | Главный скрипт проекта |
| config.json | Файл конфигурации |

Лучшие практики по оформлению README

5. Упомяни полезные ссылки

Например, ссылки на документацию, tutorиалы или демо-версию.



Общий процесс работы с Git и GitLab



1. Инициализация репозитория

- Создаём локальный репозиторий (`git init`) или клонируем удалённый репозиторий (`git clone`).
- Настраиваем `.gitignore`, чтобы исключить лишние файлы.

2. Отслеживание и фиксация изменений

- Добавляем файлы в область подготовки (`git add`).
- Фиксируем изменения (`git commit -m`).
- Просматриваем историю (`git log`).

3. Работа с ветками

- Создаём новые ветки (`git branch`).
- Переключаемся между ветками (`git switch`).
- Сливаем изменения (`git merge`).

Общий процесс работы с Git и GitLab



4. Работа с удалёнными репозиториями

- Добавляем удаленный репозиторий (git remote add).
- Отправляем изменения (git push).
- Получаем актуальную версию (git pull).

5. Настройка SSH-ключей и токенов

- Генерируем SSH-ключ и добавляем его в GitLab.
- Используем персональные токены для HTTPS.

6. Merge Requests и код-ревью

- Отправляем изменения в новую ветку.
- Создаём Merge Request.
- Проходим код-ревью и сливаем изменения.

7. Разрешение ошибок и конфликтов

- Убираем ненужные файлы (git rm --cached).
- Разрешаем конфликты (git merge + ручная правка файлов).

Общий процесс



Схематично процесс можно представить так:

[Инициализация проекта] --> [Создание веток] --> [Работа с кодом и коммиты] -->

[Отправка в GitLab] --> [Merge Request и код-ревью] --> [Слияние в основную ветку] -->

[Обновление локального репозитория] --> [Решение конфликтов при необходимости]

Общий процесс



Каждая из разобранных частей отвечает за свою часть этого процесса:

- Git даёт инструменты для локальной работы с кодом.
- GitLab позволяет организовать совместную разработку.
- SSH и токены обеспечивают безопасное взаимодействие.
- Merge Requests помогают контролировать качество кода.
- Команды для работы с ошибками позволяют исправлять проблемы.

Выводы

Git — это система контроля версий, которая позволяет отслеживать изменения, управлять ветками и работать над кодом в команде.

Есть 4 ключевых принципа работы с Git:

- История изменений
- Работа с ветками
- Распределенность
- Совместная работа



Выводы

GitLab — это платформа для управления репозиториями, которая обеспечивает хранение кода, процесс код-ревью и автоматизацию разработки.

Есть 3 ключевых элемента работы с GitLab:

- Удалённые репозитории
- Merge Requests
- SSH-ключи и токены



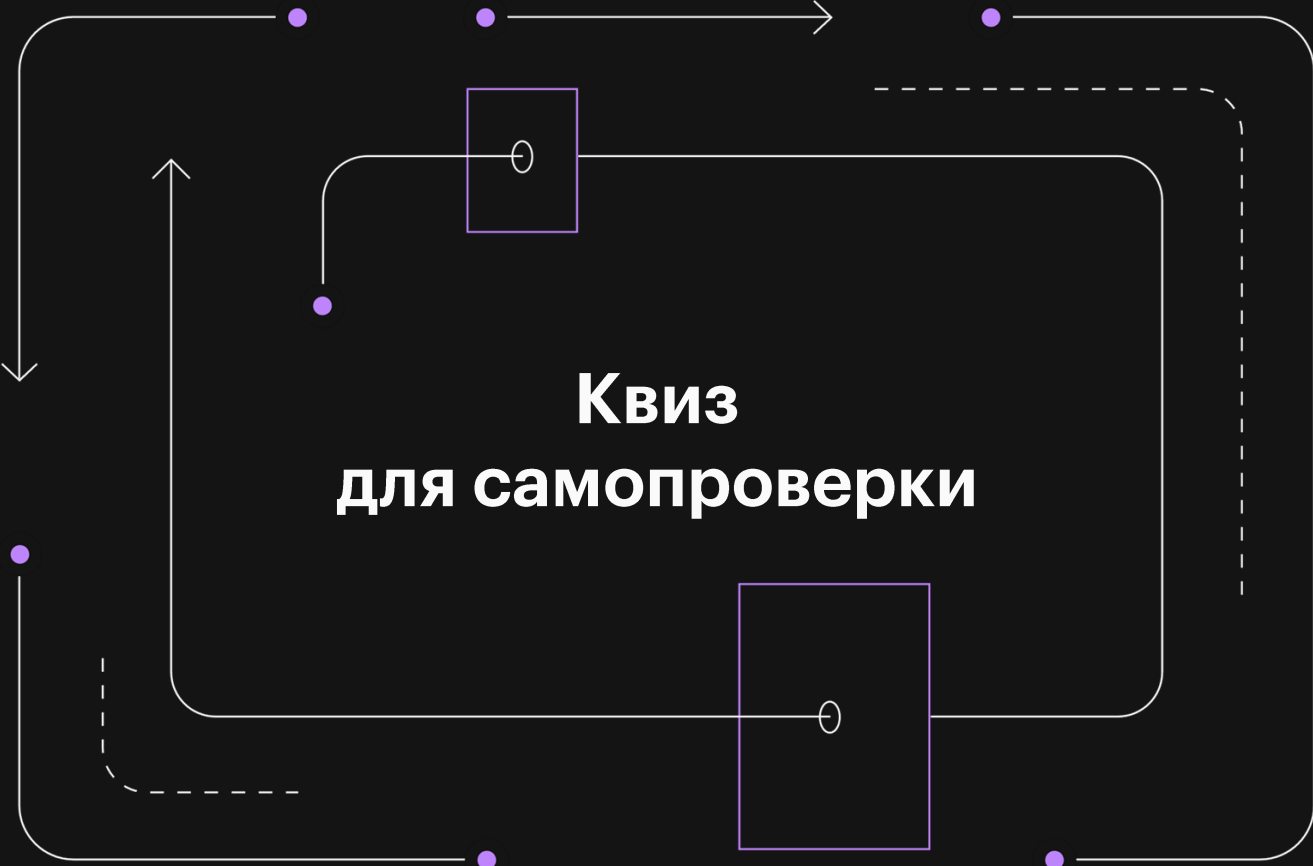
Выводы

Освоение этих инструментов позволяет гибко управлять кодом и эффективно работать в команде.



Работа с Git включает несколько этапов:

1. Создание репозитория (git init или git clone).
2. Добавление файлов и фиксация изменений (git add, git commit).
3. Работа с ветками (git branch, git merge).
4. Взаимодействие с удалённым репозиторием (git push, git pull).
5. Создание Merge Request и код-ревью.
6. Разрешение конфликтов и ошибок.



Квиз для самопроверки

Вопрос 1

Один верный ответ



Какая команда используется для создания нового коммита в Git?

1. `git add`
2. `git commit`
3. `git push`
4. `git init`

Вопрос 1

Один верный ответ



Какая команда используется для создания нового коммита в Git?

1. `git add`
2. `git commit`
3. `git push`
4. `git init`



Правильный вариант ответа: 2

Фидбэк: команда `git commit` используется для фиксации изменений в репозитории. Перед этим нужно добавить изменения в индекс с помощью `git add`. Команда `git push` отправляет изменения на удалённый сервер, а `git init` создаёт новый репозиторий.

Вопрос 2

Несколько вариантов ответов



Какие из перечисленных действий выполняются при слиянии веток в Git?

1. Создание новой ветки
2. Объединение изменений из двух веток
3. Разрешение конфликтов, если они возникают
4. Удаление одной из веток

Вопрос 2

Несколько вариантов ответов



Какие из перечисленных действий выполняются при слиянии веток в Git?

1. Создание новой ветки
2. Объединение изменений из двух веток
3. Разрешение конфликтов, если они возникают
4. Удаление одной из веток



Правильные варианты ответа: 2, 3

Фидбэк: при слиянии веток (`git merge`) изменения из одной ветки объединяются с другой. Если в одних и тех же файлах есть конфликтующие изменения, Git предложит разрешить конфликт вручную. Слияние не удаляет ветки автоматически — это нужно делать отдельно с помощью `git branch -d`.

Вопрос 3

Соответствие



Соотнесите команды Git с их описанием.

- | | |
|---------------|-----------------------------------|
| 1. git clone | A. Создаёт новый репозиторий |
| 2. git init | B. Копирует удалённый репозиторий |
| 3. git rebase | C. Перебазирует текущую ветку |
| 4. git log | D. Показывает историю коммитов |

Вопрос 3

Соответствие



Соотнесите команды Git с их описанием.

- | | |
|---------------|-----------------------------------|
| 1. git clone | A. Создаёт новый репозиторий |
| 2. git init | B. Копирует удалённый репозиторий |
| 3. git rebase | C. Перебазирует текущую ветку |
| 4. git log | D. Показывает историю коммитов |



Правильный ответ: 1 — B, 2 — A, 3 — C, 4 — D

Фидбэк:

- git clone копирует удалённый репозиторий на ваш компьютер.
- git init создаёт новый локальный репозиторий.
- git rebase перебазирует текущую ветку на другую, упорядочивая историю коммитов.
- git log показывает историю коммитов в текущей ветке.

Вопрос 4

Ранжирование



Расставьте этапы работы с Git в правильном порядке.

1. Создание коммита
2. Добавление изменений в индекс
3. Инициализация репозитория
4. Отправка изменений

Вопрос 4

Ранжирование



Расставьте этапы работы с Git в правильном порядке.

1. Создание коммита
2. Добавление изменений в индекс
3. Инициализация репозитория
4. Отправка изменений



Правильный ответ: 3, 2, 1, 4

Фидбэк: сначала нужно инициализировать репозиторий (git init), затем добавить изменения в индекс (git add), зафиксировать их (git commit) и отправить на удалённый сервер (git push).

Вопрос 5

Один верный ответ



Для чего используется Merge Request (MR) в GitLab?

1. Для создания новой ветки
2. Для отправки изменений на удалённый сервер
3. Для предложения изменений из одной ветки в другую
4. Для удаления ветки

Вопрос 5

Один верный ответ



Для чего используется Merge Request (MR) в GitLab?

1. Для создания новой ветки
2. Для отправки изменений на удалённый сервер
3. Для предложения изменений из одной ветки в другую
4. Для удаления ветки



Правильный ответ: 3

Фидбэк: Merge Request (MR) используется для предложения изменений из одной ветки в другую. Это ключевой инструмент для код-ревью и совместной работы в GitLab.

Вопрос 6

Несколько верных ответов



Какие из перечисленных действий выполняются при настройке SSH-ключей для GitLab?

1. Генерация SSH-ключа с помощью ssh-keygen
2. Копирование публичного ключа в буфер обмена
3. Добавление ключа в GitLab через раздел Settings → SSH Keys
4. Удаление существующих ключей из системы

Вопрос 6

Несколько верных ответов



Какие из перечисленных действий выполняются при настройке SSH-ключей для GitLab?

1. Генерация SSH-ключа с помощью ssh-keygen
2. Копирование публичного ключа в буфер обмена
3. Добавление ключа в GitLab через раздел Settings → SSH Keys
4. Удаление существующих ключей из системы



Правильный ответ: **1, 2, 3**

Фидбэк: для настройки SSH-ключей нужно сгенерировать ключ (ssh-keygen), скопировать публичный ключ и добавить его в GitLab через Settings → SSH Keys. Удаление существующих ключей не требуется, если они не конфликтуют.

Вопрос 7

Короткий текстовый ответ



Какой командой можно перебазировать текущую ветку на ветку main?

Вопрос 7

Короткий текстовый ответ



Какой командой можно перебазировать текущую ветку на ветку main?



Правильный ответ: `git rebase main`

Фидбэк: команда `git rebase main` перебазирует текущую ветку на ветку main, применяя изменения поверх последнего коммита в main. Это полезно для упорядочивания истории коммитов.

Вопросы

